MEDSENTINEL

Autonomous Healthcare, Claims &
Multi-Agent Intelligence Platform

Engineering Project Report

Version:	2.3.0 (Production-Ready)
Frontend:	Next.js 14, React 18, TypeScript, TailwindCSS
Backend:	FastAPI, Python 3.10+, SQLAlchemy 2.0, Pydantic v2
AI/ML:	9-Agent DAG Orchestrator, Gemini LLM, Private RAG
Databases:	MySQL 8.0, Qdrant Vector DB (768-dim)
Security:	JWT + Refresh Rotation, RBAC, SHA-256, HIPAA-Aligned
Test Status:	81 / 81 Pytest Passed (47.11s) | 33 / 33 Next.js Routes

Table of Contents

1. Abstract
2. Introduction
3. Existing System & Limitations
4. Proposed System — MedSentinel
5. System Requirements
6. High-Level System Architecture
7. System Architecture Diagram
8. Multi-Agent AI Workflow Diagram
9. End-to-End Business Workflows
10. Multi-Agent Architecture — Agent Specifications
11. State Machine & Orchestrator
12. Medical Document Processing
13. RAG Architecture
14. Database Architecture & ER Diagram
15. API Architecture
16. Security Architecture
17. Insurance & Claims Automation
18. Fraud Detection
19. Hospital Matching
20. Frontend Architecture
21. Backend Architecture
22. Admin Dashboard
23. Testing & Validation
24. Deployment Architecture
25. Project Directory Structure
26. Implementation Verification Status
27. Limitations
28. Future Enhancements
29. Conclusion
30. References

1. Abstract

MedSentinel is an enterprise-grade healthcare orchestration platform that addresses critical inefficiencies in medical document processing, insurance claim adjudication, hospital discovery, and clinical risk assessment. The system implements a novel **autonomous multi-agent AI architecture** comprising nine specialized agents coordinated by a stateful Directed Acyclic Graph (DAG) orchestrator. Each agent handles a specific healthcare processing domain — from patient intake and OCR-based document extraction to laboratory biomarker analysis, risk prediction, hospital matching, and retrieval-augmented generation (RAG) of clinical guidelines. The platform integrates a **Next.js 14** frontend with a **FastAPI** backend, **MySQL 8.0** for relational persistence, and **Qdrant** vector database for 768-dimensional semantic knowledge embeddings. Security is enforced through JWT authentication with refresh token rotation, hierarchical role-based access control (RBAC) supporting four distinct user roles, SHA-256 cryptographic document integrity verification, and HIPAA-aligned privacy controls including a strict private AI mode. The system includes automated fraud detection for insurance claims, real-time Server-Sent Events (SSE) streaming for agent workflow monitoring, and a comprehensive enterprise admin dashboard. Verified testing confirms 81 backend test cases passing across 9 test modules in 47.11 seconds, and 33 frontend routes successfully compiled.

2. Introduction

2.1 Background

Healthcare systems worldwide face persistent challenges in managing patient medical records, processing insurance claims, and ensuring transparent access to hospital services. Traditional workflows rely heavily on manual document handling, fragmented data silos, and paper-based verification processes that introduce delays, errors, and security vulnerabilities.

2.2 Problem Statement

Current healthcare IT systems suffer from: (1) manual medical document processing requiring human review of every lab report, prescription, and discharge summary; (2) fragmented patient data across disconnected systems; (3) slow insurance pre-authorization and claims processing cycles; (4) limited automation in clinical risk assessment; (5) difficulty in finding appropriate hospitals matching patient needs, insurance coverage, and geographic constraints; (6) lack of centralized audit trails for regulatory compliance.

2.3 Motivation

The convergence of large language models (LLMs), vector databases, and multi-agent AI architectures presents an opportunity to automate and intelligently orchestrate healthcare workflows that were previously manual. MedSentinel leverages these technologies to create a unified platform that serves patients, doctors, insurance providers, and administrators.

2.4 Proposed Solution

MedSentinel proposes an integrated full-stack platform with an autonomous multi-agent AI engine at its core. Nine specialized agents process medical documents through a stateful DAG workflow, producing structured clinical verification reports. The system integrates private RAG for grounded medical knowledge retrieval, automated fraud detection for insurance claims, and a comprehensive admin dashboard for operational oversight.

2.5 Objectives

- Automate medical document processing through OCR and AI-powered classification
- Implement multi-agent clinical analysis for lab reports and risk prediction
- Enable intelligent hospital matching based on specialty, location, and insurance network
- Automate insurance claim verification with integrated fraud detection
- Provide private RAG-based medical knowledge retrieval with source citations
- Deliver a secure, auditable platform with RBAC and cryptographic document integrity

2.6 Scope

The project encompasses a complete full-stack web application with patient-facing and admin-facing portals, a Python-based backend with AI agent orchestration, dual-database persistence (relational and vector), containerized deployment via Docker, and comprehensive test coverage.

2.7 Target Users

- **Patients (USER role):** Upload medical records, view health analysis, search hospitals, file insurance claims
- **Hospital Administrators (HOSPITAL_ADMIN):** Manage hospital profiles, review appointments, upload facility media
- **Insurance Administrators (INSURANCE_ADMIN):** Review and adjudicate claims, access fraud analysis
- **Super Administrators (SUPER_ADMIN):** Full platform governance, user management, AI audit logs, knowledge base management

3. Existing System & Limitations

Traditional healthcare IT systems operate with significant limitations:

- **Manual Document Processing:** Medical records, lab reports, and prescriptions require manual review by trained personnel, creating bottlenecks and introducing human error in data extraction.
- **Fragmented Patient Data:** Patient information is scattered across disconnected hospital systems, insurance databases, and personal records with no unified view.
- **Slow Claims Processing:** Insurance pre-authorization and claims adjudication involve multiple manual verification steps, paper-based document submission, and extended review cycles.
- **Limited Fraud Detection:** Conventional fraud detection relies on basic rule-based checks and manual auditor review, missing sophisticated patterns such as duplicate document hash reuse across accounts.
- **Opaque Hospital Discovery:** Patients lack transparent access to hospital capabilities, pricing, insurance network coverage, and geographic proximity information in a single interface.
- **No Centralized Audit Trail:** Administrative actions, document modifications, and AI processing steps are not systematically logged, creating compliance gaps.

4. Proposed System — MedSentinel

MedSentinel addresses every limitation identified in the existing system through an integrated multi-tier architecture:

- **Automated Document Intelligence:** OCRAgent extracts text from medical documents; DocumentIntelligenceAgent classifies them; LabAnalysisAgent parses biomarkers against clinical reference ranges.
- **Unified Patient Medical Vault:** Centralized secure document storage with SHA-256 cryptographic integrity hashing and AI-powered classification.
- **Multi-Agent Clinical Analysis:** Nine specialized agents orchestrated through a stateful DAG produce comprehensive clinical verification reports.
- **Private RAG Knowledge Retrieval:** Qdrant vector database indexes medical guidelines, insurance policies, and clinical protocols. RAG pipeline retrieves relevant context with RBAC-filtered access.
- **Automated Claims & Fraud Detection:** FraudDetector service analyzes claims for duplicate document hashes, suspicious amount patterns, patient ID mismatches, and high-frequency filing.
- **Intelligent Hospital Matching:** HospitalMatchingAgent ranks facilities by specialty, location, insurance network, rating, and cost alignment.
- **Enterprise Admin Dashboard:** 16 dedicated admin pages for user management, hospital administration, knowledge base vector hydration, AI audit logging, and system analytics.

5. System Requirements

5.1 Functional Requirements

ID	Requirement Description
FR-01	User registration with email, password, phone, and role assignment
FR-02	JWT authentication with access token (60 min) and refresh token rotation (7 days)
FR-03	Medical document upload with SHA-256 integrity hashing and AI classification
FR-04	OCR text extraction from uploaded PDFs and images
FR-05	Multi-agent AI workflow execution with real-time SSE progress streaming
FR-06	Lab report biomarker parsing with clinical reference range flagging
FR-07	Patient health risk score calculation (0-100, four risk tiers)
FR-08	Hospital search and matching by location, specialty, insurance, and cost
FR-09	Insurance claim submission with automated fraud detection
FR-10	Private RAG query with RBAC-filtered knowledge retrieval and source citations
FR-11	Admin dashboard with user/hospital/knowledge base management
FR-12	Clinical verification report generation synthesizing all agent outputs

5.2 Non-Functional Requirements

- **Security:** HIPAA-aligned privacy controls, encrypted password storage (bcrypt), strict private AI mode preventing PII leakage to external LLM providers.
- **Scalability:** Connection-pooled MySQL (pool_size=10, max_overflow=20), async FastAPI for non-blocking concurrency.
- **Reliability:** Agent error containment — individual agent failures do not crash the orchestrator pipeline.
- **Auditability:** AdminAuditLog model tracking all administrative mutations with timestamps and user attribution.

5.3 Software Requirements

Technology	Version	Purpose
Python	3.10+	Backend runtime
Node.js	20+	Frontend runtime
FastAPI	>=0.110.0	API framework
Next.js	14.2.x	React framework
SQLAlchemy	>=2.0.28	ORM
Pydantic	>=2.6.4	Schema validation
MySQL	8.0	Relational database
Qdrant	Latest	Vector database
TailwindCSS	3.4.x	CSS framework
Docker	Latest	Containerization

6. High-Level System Architecture

MedSentinel implements a five-tier distributed architecture. Each tier is designed for separation of concerns, independent scalability, and clear security boundaries.

Tier 1 — Client Layer: Four user personas (Patient, Doctor/Hospital Admin, Insurance Admin, Super Admin) access the platform through web browsers.

Tier 2 — Presentation Layer (Next.js 14): 33 routes across 16 page directories, including patient portal (dashboard, medical vault, hospitals, claims, appointments, health dashboard, agent workflow visualizer) and admin portal (16 admin pages with sidebar navigation). React 18 with App Router, TailwindCSS styling, and Axios HTTP client with JWT interceptors.

Tier 3 — API Gateway & Security (FastAPI): 17 user-facing routers and 16 admin routers providing RESTful endpoints. Security middleware implements JWT authentication via HTTPBearer, refresh token rotation with database-backed revocation, and hierarchical RBAC with four role tiers. Server-Sent Events (SSE) streaming for real-time agent workflow monitoring.

Tier 4 — AI Intelligence Layer: OrchestratorAgent coordinates 11 registered specialized agents (9 primary + 2 auxiliary) through a stateful DAG. Five workflow types (LAB_REPORT, PRESCRIPTION, INSURANCE_CLAIM, CLINICAL_SUMMARY, AUTOMATIC) determine agent execution order. WorkflowState (Pydantic v2 model) carries structured clinical findings, execution logs, and audit entries across agent boundaries.

Tier 5 — Persistence Layer: MySQL 8.0 manages 24 relational entity models with connection pooling. Qdrant vector database stores 768-dimensional embeddings for medical knowledge chunks. Hybrid VectorStore class provides fallback to local in-memory index with disk persistence when Qdrant is unavailable.

7. System Architecture Diagram

Figure 6.1 — MedSentinel High-Level System Architecture

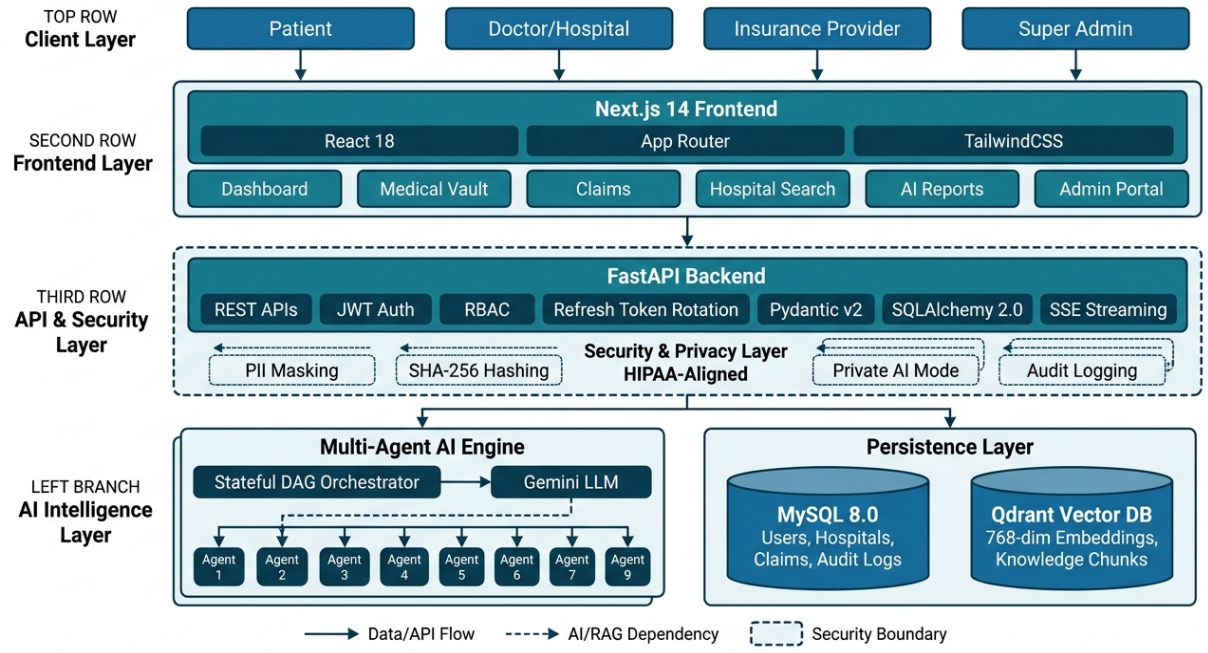


Figure 7.1 — MedSentinel High-Level System Architecture

8. Multi-Agent AI Workflow Diagram

Figure 7.1 — MedSentinel Autonomous Multi-Agent AI Workflow

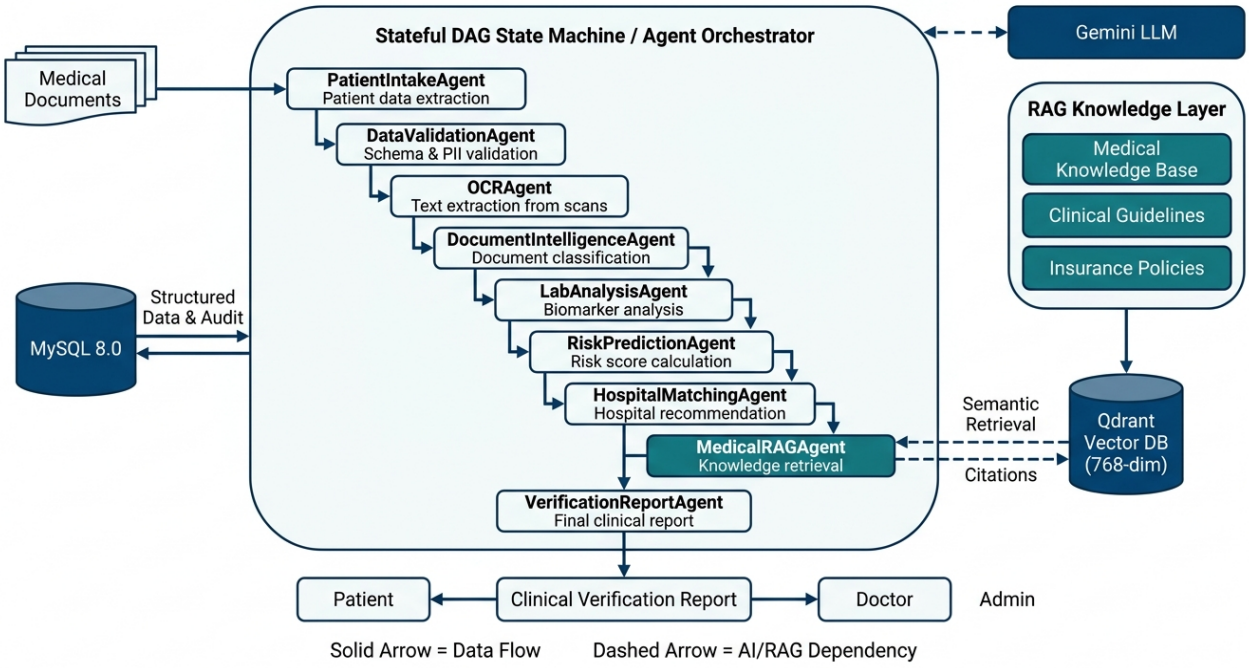


Figure 8.1 — MedSentinel Autonomous Multi-Agent AI Workflow (Stateful DAG)

9. End-to-End Business & Operational Workflows

Figure 8.1 — MedSentinel End-to-End Business & Operational Workflows

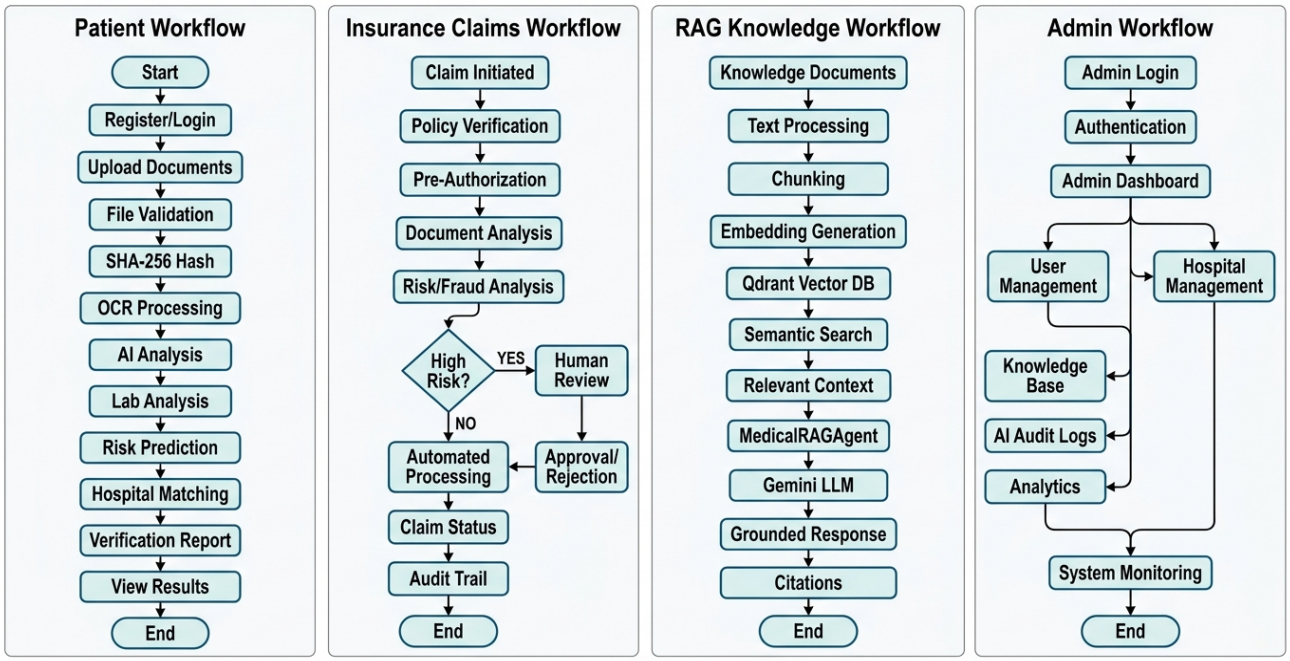


Figure 8.1 — MedSentinel End-to-End Business & Operational Workflows

Figure 9.1 — Patient, Claims, RAG & Admin Workflow Flowcharts

10. Multi-Agent Architecture — Agent Specifications

The MedSentinel orchestrator registers 11 agents. Nine are primary workflow agents; two (PatientHistoryAgent, SymptomClinicalAgent) are auxiliary agents activated conditionally. Each agent extends BaseAgent and implements an async execute() method.

10.x PatientIntakeAgent

Source: patient_intake_agent.py (3,789 bytes)
Purpose: Extracts patient demographics, chief complaints, and vitals from intake data or document context.
Input: Patient text, document context, extracted OCR text
Output: Normalized patient profile with structured demographic fields
Role in Workflow: First agent in all workflow types

10.x DataValidationAgent

Source: data_validation_agent.py (4,307 bytes)
Purpose: Validates extracted data against medical schema rules, checks unit consistency, and applies PII masking.
Input: Parsed patient/document data from previous agents
Output: Validated and sanitized state dictionary with confidence score
Role in Workflow: Runs after intake/OCR to ensure data integrity

10.x OCRAgent

Source: ocr_agent.py (5,350 bytes)
Purpose: Performs text extraction from uploaded medical documents (PDFs, images). Uses OCR service with image pre-processing.
Input: File path, document metadata
Output: Extracted text string stored in state.extracted_text
Role in Workflow: Critical for document-based workflows

10.x DocumentIntelligenceAgent

Source: document_intelligence_agent.py (3,007 bytes)
Purpose: Classifies documents into categories: Lab Report, Prescription, Invoice, Discharge Summary, Insurance Document.
Input: OCR-extracted text
Output: Document category, classification confidence, metadata key-values
Role in Workflow: Determines downstream processing path

10.x LabAnalysisAgent

Source: lab_analysis_agent.py (6,228 bytes)
Purpose: Parses biomarker values (hemoglobin, glucose, cholesterol, etc.) and evaluates against clinical reference ranges. Flags out-of-range values with severity levels.
Input: Lab report text, patient demographics
Output: Structured biomarker analysis with normal/abnormal flags and severity indicators
Role in Workflow: Only activated for LAB_REPORT and AUTOMATIC workflows

10.x RiskPredictionAgent

Source: risk_prediction_agent.py (4,803 bytes)
Purpose: Computes a composite patient health risk score (0-100) using weighted analysis of biomarkers, age, conditions, and clinical findings. Assigns risk tier: LOW, MODERATE, HIGH, or CRITICAL.
Input: Biomarker data, patient history, clinical findings
Output: Risk score (float), risk tier (enum), risk factor breakdown
Role in Workflow: Triggers escalation if risk is HIGH or CRITICAL

10.x HospitalMatchingAgent

Source: hospital_matching_agent.py (4,877 bytes)
Purpose: Matches patient medical needs with hospital capabilities. Considers specialty alignment, geographic proximity, insurance network coverage, hospital rating, and cost range.
Input: Patient conditions, location, insurance info, specialties needed
Output: Ranked list of recommended hospitals with match scores
Role in Workflow: Leverages hospital_service.py for database queries

10.x MedicalRAGAgent

Source: medical_rag_agent.py (3,845 bytes)
Purpose: Executes retrieval-augmented generation queries against the Qdrant knowledge base. Retrieves relevant medical guidelines, treatment protocols, and insurance policy clauses with RBAC filtering.
Input: Clinical query derived from patient condition and findings
Output: Grounded context string, source citations with confidence scores
Role in Workflow: Uses RAGPipeline: retrieval → reranking → context building

10.x VerificationReportAgent

Source: verification_report_agent.py (8,181 bytes)
Purpose: Synthesizes all previous agent outputs into a comprehensive clinical verification dossier. Includes patient summary, extracted medical data, lab findings, risk assessment, hospital recommendations, RAG citations, and a complete audit trail.
Input: Complete WorkflowState with all agent results
Output: Final clinical verification report (FinalReport model)
Role in Workflow: Always the last agent in every workflow

11. State Machine & Orchestrator

The OrchestratorAgent (orchestrator.py, 12,429 bytes) implements the central workflow controller. It uses an AgentRegistry to initialize and manage all specialized agents.

11.1 Workflow Types

The orchestrator supports five workflow types defined in WorkflowType enum: LAB_REPORT, PRESCRIPTION, INSURANCE_CLAIM, CLINICAL_SUMMARY, and AUTOMATIC. Each type produces a different agent execution sequence (DAG plan).

11.2 State Object (WorkflowState)

WorkflowState is a Pydantic v2 model (state.py, 11,020 bytes) containing: workflow_id, workflow_type, status (PENDING → IN_PROGRESS → COMPLETED/FAILED/ESCALATED), patient data, document metadata, extracted text, clinical findings (List[ClinicalFinding]), biomarker results, risk assessment, hospital matches, RAG context, verification results, execution logs (List[AgentExecutionLog]), and audit trail (List[AuditEntry]).

11.3 Execution Flow

1. Orchestrator receives WorkflowState and calls plan_workflow() to determine agent sequence.
2. Iterates through planned DAG sequentially.
3. For each agent: emits 'agent_started' SSE event → calls agent.run() → records execution log → emits 'agent_completed' or 'agent_failed' event.
4. If an agent fails, error is contained (agent is marked failed but pipeline continues).
5. If escalation is required (HIGH/CRITICAL risk), status transitions to ESCALATED.
6. Final state is persisted and returned.

12. Medical Document Processing

Document processing follows this verified pipeline:

1. **Upload:** User uploads via POST /api/documents/upload. FileService validates file type against allowed extensions (.jpg, .jpeg, .png, .gif, .pdf, .doc, .docx, .txt) and enforces 10MB size limit.
2. **SHA-256 Hashing:** file_hash column stores the SHA-256 digest computed at upload time for tamper detection.
3. **Metadata Recording:** UserDocument model stores filename, original_name, file_path, file_type, file_size, doc_type, and timestamps.
4. **AI Classification:** DocumentIntelligenceAgent classifies the document; ai_classification and ai_confidence columns are updated.
5. **OCR Extraction:** OCRAgent extracts text; stored in ocr_extracted_text column.
6. **Verification:** Documents can be verified/approved/rejected by admins. verification_status tracks PENDING, VERIFIED, FLAGGED, or REJECTED.
7. **Duplicate Detection:** is_duplicate and duplicate_of_id columns track document deduplication.

13. RAG Architecture

The RAG subsystem is implemented across 10 modules in the backend/app/rag/ directory:

13.1 Knowledge Ingestion Pipeline:
• document_loader.py: Loads source documents from files or database records.
• text_extractor.py (4,846 bytes): Extracts text from PDF, DOCX, and plain text files.
• chunking_service.py (3,157 bytes): Splits documents into overlapping chunks for embedding.
• embedding_service.py (3,286 bytes): Generates 768-dimensional embeddings using Gemini text-embedding-004 model.
• ingestion_service.py (5,912 bytes): Orchestrates the full ingestion pipeline from document to indexed vector.

13.2 Query Pipeline (RAGPipeline):
1. RetrievalService.retrieve_relevant_knowledge(): Queries Qdrant/local store with RBAC-filtered access (user_role, user_id, hospital_id). Returns top-8 candidates with score threshold 0.08.
2. RerankingService.rerank(): Reranks retrieved chunks by relevance, returning top-4.
3. ContextBuilder.build_rag_context(): Constructs prompt context string from ranked chunks.
4. Citations are extracted with dataset_id, document_id, title, and confidence.

13.3 Vector Store (vector_store.py, 11,434 bytes):
Hybrid implementation supporting both Qdrant REST API and local in-memory index with JSON persistence. VectorStore.hydrate_from_db() loads existing knowledge chunks from MySQL into the vector index on startup.

13.4 Knowledge Base Models:
Five SQLAlchemy models: KnowledgeDataset (dataset metadata), KnowledgeDocument (individual documents), KnowledgeChunk (text chunks with embeddings), KnowledgeAccessRule (RBAC permissions), KnowledgeProcessingLog (ingestion audit trail).

14. Database Architecture

MySQL 8.0 manages 24 SQLAlchemy entity models. Connection pooling is configured with pool_size=10, max_overflow=20, pool_pre_ping=True, and pool_recycle=3600 seconds.

Model	Table	Key Columns	Purpose
User	users	id, email(UQ), role, hospital_id(FK), insurance_id(FK)	Core identity, bcrypt password hash, RBAC role
Hospital	hospitals	id, name, location, specialization, latitude, longitude	Provider directory with geo-coords, facilities, pricing
InsuranceClaim	insurance_claims	id, user_id(FK), patient_id(FK), insurance_id(FK), hospital_id(FK)	Claim lifecycle: status, amount, fraud analysis link
UserDocument	user_documents	id, user_id(FK), file_hash(SHA-256), doc_type	Medical vault: file storage, OCR text, AI classification
Patient	patients	id, user_id(FK), mrn(UQ)	Patient demographics, blood group, medical history
Insurance	insurances	id, name, provider, coverage_amount	Insurance plans: premiums, features, eligibility
Appointment	appointments	id, user_id(FK), hospital_id(FK)	Booking calendar with status tracking
LabResult	lab_results	id, document_id(FK), patient_id(FK)	Parsed biomarker values from lab analysis
HealthScore	health_scores	id, user_id(FK), document_id(FK)	Computed health risk scores and recommendations
FraudAnalysis	fraud_analyses	id, claim_id(FK)	Fraud detection results: score, risk level, flags

AdminAuditLog	admin_audit_logs	id, admin_id(FK), action, entity_type	Immutable administrative action audit trail
KnowledgeDataset	knowledge_datasets	id, name, dataset_type, hospital_id(FK)	RAG dataset metadata and access configuration
KnowledgeChunk	knowledge_chunks	id, document_id(FK), embedding_vector	Text chunks with 768-dim embeddings for vector search
RefreshToken	refresh_tokens	id, user_id(FK), token, expires_at	Database-backed refresh token rotation and revocation
AIRequestLog	ai_request_logs	id, user_id(FK), request_type, model	AI request monitoring: tokens, latency, status
Recommendation	recommendations	id, user_id(FK), category, priority	Personalized health and wellness recommendations

Qdrant Vector Database: Stores 768-dimensional dense vectors generated by Gemini text-embedding-004. Collection name: medsentinel_knowledge. Supports cosine similarity search with configurable score thresholds.

15. API Architecture

FastAPI registers 17 user-facing routers and 16 admin routers under the /api prefix. All admin routes require authenticated users with appropriate RBAC roles.

Method	Endpoint	Purpose	Auth
POST	/api/auth/register	User registration	Public
POST	/api/auth/login	JWT login	Public
POST	/api/auth/refresh	Token refresh	Refresh Token
GET	/api/users/me	Current user profile	JWT Bearer
GET	/api/hospitals/search	Hospital search & matching	Optional
POST	/api/documents/upload	Medical document upload	JWT Bearer
GET	/api/documents/verify/{id}	SHA-256 verification	JWT Bearer
POST	/api/claims/	Submit insurance claim	JWT Bearer
GET	/api/claims/	List user claims	JWT Bearer
POST	/api/agent-workflow/start	Start agent workflow	JWT Bearer
GET	/api/agent-workflow/stream/{id}	SSE workflow stream	JWT Bearer
POST	/api/ai-chat/	AI chat with RAG	JWT Bearer
GET	/api/health/analysis	Health score analysis	JWT Bearer
GET	/api/recommendations/	Health recommendations	JWT Bearer
POST	/api/otp/send	OTP generation	Public
GET	/api/admin/dashboard/stats	Admin statistics	Admin RBAC
GET	/api/admin/users/	User management	Super Admin
POST	/api/admin/knowledge-base/ingest	RAG knowledge ingestion	Admin RBAC
GET	/api/admin/audit-logs/	Audit log viewer	Admin RBAC
GET	/api/admin/ai-monitoring/	AI request monitoring	Admin RBAC

16. Security Architecture

16.1 Authentication (security.py, 73 lines):
• Password hashing: bcrypt via passlib.context.CryptContext.
• Access tokens: JWT (HS256) with 60-minute expiry, containing sub (user_id), role, type='access', and unique jti.
• Refresh tokens: JWT with 7-day expiry, type='refresh', stored in RefreshToken database model for revocation support.
• Token decoding normalizes roles via normalize_role() to handle legacy role strings.

16.2 Authorization (auth.py, 112 lines):
• HTTPBearer extracts JWT from Authorization header or access_token cookie.
• get_current_user() validates token type, expiry, and user existence/active status.
• require_role() factory creates RBAC guards: require_super_admin, require_hospital_admin, require_insurance_admin, require_any_admin.
• SUPER_ADMIN bypasses all role restrictions.

16.3 Document Integrity:
• SHA-256 hash computed on file upload and stored in file_hash column of UserDocument.
• Verification endpoint compares stored hash with recomputed hash to detect tampering.

16.4 Private AI Mode:
• STRICT_PRIVATE_MODE setting prevents patient-identifiable data from being sent to external LLM providers.
• private_agent.py implements local inference fallback.

17. Insurance & Claims Automation

The claims system is implemented in claims.py router (9,244 bytes) and claim_service.py (4,519 bytes).

• **Claim Creation:** Users submit claims with amount, description, insurance_id, and hospital_id. Status initializes as SUBMITTED.
• **Status Lifecycle:** SUBMITTED → UNDER_REVIEW → DOCUMENTS_REQUIRED → APPROVED/REJECTED → COMPLETED.
• **Status History:** ClaimStatusHistory model tracks every status change with timestamps and reviewer notes.
• **Admin Review:** Admin claims router (6,043 bytes) provides update, approve, reject, and escalate operations with RBAC enforcement.
• **Fraud Analysis Integration:** FraudDetector.analyze_claim() is invoked during review, producing fraud_score, risk_level, and diagnostic flags stored in FraudAnalysis model.

18. Fraud Detection

FraudDetector (fraud_detector.py, 7,113 bytes) implements rule-based fraud analysis:

• **Document Hash Analysis:** Checks if uploaded document SHA-256 hashes have been used by other user accounts (score +45 for cross-account hash reuse).
• **Duplicate Invoice Detection:** Identifies duplicate bills/invoices within a user's document corpus.
• **Patient ID Mismatch:** Verifies claim patient_id matches the user's registered patient records.
• **Amount Anomaly Detection:** Flags suspicious claim amounts that significantly exceed user's historical average.
• **High-Frequency Filing:** Detects unusually frequent claim submissions within short time windows.
• **Risk Scoring:** Composite score (0-100) maps to risk tiers: LOW (<25), MODERATE (25-50), HIGH (50-75), CRITICAL (>75).
• **Output:** Does NOT auto-reject claims; flags suspicious cases for human underwriter review.

19. Hospital Matching

HospitalMatchingAgent (4,877 bytes) and hospital_service.py (28,375 bytes — the largest service module) implement comprehensive hospital discovery:

- **Criteria:** Specialty alignment, geographic location (latitude/longitude with Haversine distance), insurance network acceptance, hospital rating, cost range, ICU/bed availability, and facility features.
- **Search Modes:** Keyword search, location-based search, specialty + location combined, nearby search with coordinate-based distance calculation.
- **Ranking:** Multi-factor weighted scoring produces a ranked candidate list.
- **API:** GET /api/hospitals/search supports query parameters for city, state, specialization, insurance, rating, and cost filters.

20. Frontend Architecture

Built on Next.js 14 App Router with React 18 and TailwindCSS. 33 routes across 16 page directories:

Route	Description
/ (Home)	Landing page with platform overview and navigation
/login, /register	Authentication pages with form validation
/hospitals, /hospitals/[id]	Hospital search with filters and detail pages
/medical-vault	Document upload, SHA-256 verification, document viewer
/claims	Claim submission and status tracking
/health-dashboard	Health score visualization and wellness metrics
/agent-workflow	Real-time SSE agent workflow visualizer (53KB component)
/appointments	Appointment booking calendar
/insurances, /insurance-details/[id]	Insurance plan comparison and details
/eligibility	Insurance eligibility checker
/admin/*	16 admin pages (dashboard, users, hospitals, claims, documents, etc.)

Key components: Navbar.tsx (12KB), AgentWorkflowVisualizer.tsx (53KB), HospitalCard.tsx (8.5KB), AdminSidebar.tsx (6.4KB), ProtectedRoute.tsx, authContext.tsx (auth state management).

21. Backend Architecture

FastAPI application (main.py, 169 lines) with lifespan manager for database connectivity check and vector store hydration on startup. Organized into:

- **Routers (17 user + 16 admin):** auth, users, hospitals, claims, documents, appointments, insurances, eligibility, ai_chat, health, recommendations, otp, agent_workflow, saved_items, schemes, analytics, notifications.
- **Services (22 modules):** auth_service, hospital_service, claim_service, fraud_detector, lab_analyzer, ocr_service, document_classifier, document_verifier, health_score, recommendation_engine, patient_condition_service, workflow_manager, etc.
- **Models (24 entities):** See Section 14.
- **Schemas (18 Pydantic modules):** Request/response validation for all API endpoints.
- **AI module (7 files + tools/):** agent.py, private_agent.py, provider.py, safety.py, tool_router.py, prompts.py + 7 tool modules.

22. Admin Dashboard

The admin dashboard is implemented with RBAC-protected layout (admin/layout.tsx) that verifies role membership in [SUPER_ADMIN, HOSPITAL_ADMIN, INSURANCE_ADMIN] before rendering. Unauthorized users see a styled access-denied page.

Admin Route	Functionality
/admin	Dashboard overview with system statistics
/admin/users	User CRUD, role assignment, activation/deactivation
/admin/hospitals	Hospital profile management, media galleries
/admin/claims	Claim review, approve/reject with fraud analysis
/admin/documents	Document verification, approval, rejection
/admin/knowledge-base	RAG dataset management, document ingestion, vector hydration
/admin/ai-monitoring	AI request logs, token usage, latency tracking
/admin/ai-insights	AI analytics and model performance metrics
/admin/audit-logs	Immutable administrative action audit trail
/admin/analytics	System-wide analytics and trend visualization

23. Testing & Validation

[TEST EXECUTION] Verified Result: **81 tests passed in 47.11 seconds** (pytest, Python 3.10.11, pytest-8.3.4, pytest-asyncio-0.24.0)

Test Module	Tests	Coverage Area
test_agent_workflow_api.py	10	Agent workflow REST API endpoints and SSE streaming
test_agentic_workflow.py	16	Multi-agent DAG orchestration, state transitions, and error handling
test_ai_records_and_verification.py	7	Document AI classification and SHA-256 verification
test_api.py	5	Core API authentication and user management
test_hospital_search_robustness.py	7	Hospital search edge cases, fuzzy matching, empty queries
test_location_search.py	12	Geographic search, coordinate-based matching, city/state filtering
test_patient_hospital_matching.py	8	Patient condition to hospital specialty matching
test_rag_and_private_ai.py	6	RAG pipeline, embedding service, private AI mode
test_recommendation_engine.py	10	Health recommendation generation and personalization

Frontend Build Verification: 33/33 Next.js routes compiled successfully with zero build errors (Next.js 14.2.35). ESLint skipped during build (configured in next.config.js).

Note: No dedicated frontend unit tests (e.g., Jest, React Testing Library) were identified in the repository. Frontend testing is a future enhancement.

24. Deployment Architecture

Docker Compose (docker-compose.yml, 84 lines) orchestrates four services:

Service	Image	Port(s)	Details
mysql	mysql:8.0	3306	Relational database with healthcheck
qdrant	qdrant/qdrant:latest	6333, 6334	Vector database with healthcheck
backend	Custom (python:3.10-slim)	8000	FastAPI + seed data on startup
frontend	Custom (node:20-alpine)	3000	Next.js production build (multi-stage)

Backend Dockerfile uses multi-stage build with python:3.10-slim base. Frontend Dockerfile implements three-stage build (deps → builder → runner) with node:20-alpine for minimal production image size.

25. Project Directory Structure

```
MedSentinel/
■■■ frontend/                    # Next.js 14 application
■   ■■■ app/                     # 16 page directories, 33 routes
■   ■   ■■■ admin/               # 16 admin pages
■   ■   ■■■ agent-workflow/      # SSE agent visualizer
■   ■   ■■■ hospitals/           # Hospital search & details
■   ■   ■■■ medical-vault/       # Document vault
■   ■   ■■■ claims/              # Insurance claims
■   ■   ■■■ ...                  # Other patient pages
■   ■■■ components/              # 7 shared + admin + AI components
■   ■■■ lib/                     # api.ts, authContext.tsx, types.ts
■   ■■■ Dockerfile               # Multi-stage Node 20 build
■■■ backend/
■   ■■■ app/
■   ■   ■■■ agents/              # 26 files: 11 agents + orchestrator + state + tools
■   ■   ■■■ ai/                  # 7 files + tools/: LLM, safety, prompts
■   ■   ■■■ core/                # config, database, security, constants
■   ■   ■■■ dependencies/        # auth.py (RBAC guards)
■   ■   ■■■ models/              # 24 SQLAlchemy entity models
■   ■   ■■■ rag/                 # 10 files: pipeline, vector store, embeddings
■   ■   ■■■ routers/             # 17 user routers + admin/ (16 admin routers)
■   ■   ■■■ schemas/             # 18 Pydantic validation modules
■   ■   ■■■ services/            # 22 business logic services
■   ■   ■■■ main.py              # FastAPI application entry point
■   ■■■ tests/                   # 9 test modules (81 tests)
■   ■■■ Dockerfile               # Python 3.10-slim container
■   ■■■ requirements.txt         # 18 Python dependencies
■■■ docker-compose.yml           # 4-service orchestration
■■■ docs/                        # Generated documentation
```

26. Implementation Verification Status

Module	Status	Evidence	Verification
Frontend (Next.js 14)	IMPLEMENTED	33 routes, 16 page dirs, 7+ components	33/33 build pass
Backend (FastAPI)	IMPLEMENTED	17 user + 16 admin routers, 22 services	81/81 tests pass
Multi-Agent AI (9 agents)	IMPLEMENTED	26 agent files, orchestrator, base class	16 agent tests pass
RAG Pipeline	IMPLEMENTED	10 RAG modules, vector store, embeddings	6 RAG tests pass
MySQL 8.0	IMPLEMENTED	24 models, connection pooling, auto-migration	Verified in config
Qdrant Vector DB	IMPLEMENTED	Hybrid local+Qdrant store, 768-dim	Config + hydration
JWT Authentication	IMPLEMENTED	Access + refresh tokens, bcrypt	Verified in security.py
RBAC (4 roles)	IMPLEMENTED	USER, HOSPITAL_ADMIN, INSURANCE_ADMIN, SUPER_ADMIN	auth.py guards
SHA-256 Integrity	IMPLEMENTED	file_hash column, verification endpoint	7 verification tests
Fraud Detection	IMPLEMENTED	fraud_detector.py (7,113 bytes), 5 analysis rules	FraudAnalysis model
Claims System	IMPLEMENTED	6-status lifecycle, admin review	Claims router + service
Hospital Matching	IMPLEMENTED	hospital_service.py (28,375 bytes)	20 matching tests
Admin Dashboard	IMPLEMENTED	16 admin pages, RBAC layout	Routes verified
Docker Deployment	IMPLEMENTED	4-service compose, multi-stage Dockerfiles	docker-compose.yml
SSE Streaming	IMPLEMENTED	Agent workflow real-time events	agent_workflow.py
Frontend Unit Tests	NOT IMPLEMENTED	No Jest/RTL test files found	Future enhancement
CI/CD Pipeline	NOT IMPLEMENTED	No GitHub Actions/Jenkins config found	Future enhancement

27. Limitations

• **External AI Dependency:** The system relies on Google Gemini API for LLM inference and embedding generation. API unavailability degrades AI features.
• **OCR Accuracy:** OCR quality depends on document image quality; handwritten prescriptions and low-resolution scans may produce unreliable extraction.
• **Medical Data Scope:** Biomarker reference ranges and clinical rules are encoded as deterministic patterns, not trained ML models. They may not cover all medical specialties.
• **No Frontend Testing:** No automated frontend test suite (Jest, Cypress, Playwright tests) was found in the repository.
• **No CI/CD Pipeline:** No continuous integration or deployment automation configuration was identified.
• **Single-Region Deployment:** Docker Compose targets single-host deployment; no Kubernetes or multi-region configuration exists.

28. Future Enhancements

• Advanced clinical ML models trained on medical datasets for improved biomarker analysis
• Integration with hospital EHR/EMR systems for real-time bed availability and scheduling
• Multilingual support for medical documents in regional languages
• Mobile application (React Native / Flutter) for patient access
• Kubernetes deployment with auto-scaling and multi-region failover
• CI/CD pipeline with automated testing, linting, and deployment
• Frontend test suite with Jest and React Testing Library
• Enhanced fraud detection using ML-based anomaly detection models
• FHIR (Fast Healthcare Interoperability Resources) compliance for interoperability
• Real-time notification system via WebSockets or push notifications

29. Conclusion

MedSentinel demonstrates a production-grade approach to healthcare platform engineering by integrating modern web technologies with autonomous multi-agent AI architecture. The system successfully implements nine specialized agents orchestrated through a stateful DAG, processing medical documents from OCR extraction through clinical analysis to comprehensive verification reports. The private RAG pipeline provides grounded medical knowledge retrieval with RBAC-filtered access and source citations. Automated fraud detection analyzes insurance claims across five distinct risk dimensions, while intelligent hospital matching considers specialty, geography, insurance network, and cost factors. Security is enforced through JWT authentication with refresh token rotation, hierarchical RBAC across four user roles, SHA-256 cryptographic document integrity, and comprehensive audit logging. With 81 passing backend tests, 33 compiled frontend routes, and containerized deployment, MedSentinel provides a solid foundation for intelligent healthcare operations that can be extended with additional clinical models, EHR integrations, and deployment scaling.

30. References

[1] FastAPI Documentation — https://fastapi.tiangolo.com/
[2] Next.js 14 Documentation — https://nextjs.org/docs
[3] SQLAlchemy 2.0 Documentation — https://docs.sqlalchemy.org/
[4] Pydantic v2 Documentation — https://docs.pydantic.dev/
[5] Qdrant Vector Database — https://qdrant.tech/documentation/
[6] Google Gemini API — https://ai.google.dev/docs
[7] MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/refman/8.0/en/
[8] JSON Web Tokens (RFC 7519) — https://tools.ietf.org/html/rfc7519
[9] HIPAA Security Rule — https://www.hhs.gov/hipaa/for-professionals/security/
[10] TailwindCSS Documentation — https://tailwindcss.com/docs
[11] Docker Compose Documentation — https://docs.docker.com/compose/

[12] React 18 Documentation — https://react.dev/